Combinational Logic

Chapter 4

Analysis, design, arithmetic circuits, data selectors, and Verilog HDL

Mano & Ciletti, *Digital Design*, 5th edition

Learning objectives

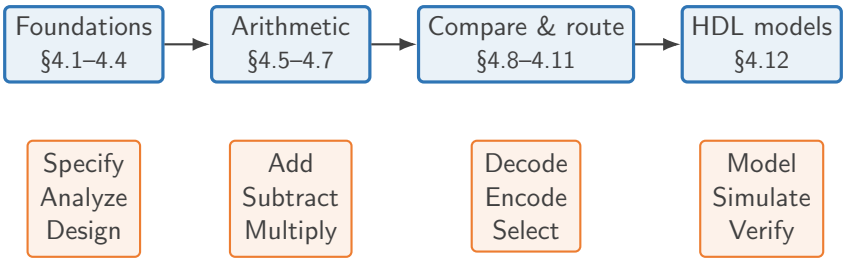
By the end of this chapter, you should be able to:

- distinguish **combinational** logic from sequential logic;
- analyze a circuit from its diagram and synthesize one from a specification;
- design and connect standard blocks: adders, subtractors, comparators, decoders, encoders, and multiplexers;
- explain propagation delay, carry lookahead, BCD correction, and overflow;
- describe and verify combinational logic using **Verilog HDL**.

Core idea

Every output is a Boolean function of the *present* inputs only.

Chapter roadmap



Combinational versus sequential logic

Combinational circuit

$$\mathbf{Y}(t) = F(\mathbf{X}(t))$$

- no memory elements;
- no feedback path carrying state;
- output follows the current input after propagation delay.

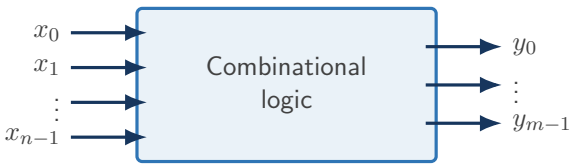
Sequential circuit

$$\mathbf{Y}(t) = G(\mathbf{X}(t), \mathbf{Q}(t))$$

- contains storage;
- state depends on earlier inputs;
- behavior must be described over time.

First analysis check: look for memory or feedback before treating a circuit as combinational.

The combinational-circuit abstraction



For n binary inputs:

- there are 2^n input combinations;
- each of the m outputs has its own Boolean function;
- the complete behavior is captured by a truth table.

$$y_j = f_j(x_0, \ldots, x_{n-1})$$

Three equivalent descriptions

Truth table

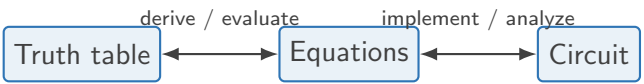
Exhaustive behavioral specification for all 2^n input patterns.

Boolean equations

Algebraic description that can be simplified and factored.

Logic diagram

Structural implementation using gates or standard components.



Simulation checks the implemented circuit against its specified behavior.

Analysis procedure: circuit → behavior

1. Verify that the circuit has no storage elements or feedback paths.
2. Label outputs of gates driven directly by the primary inputs.
3. Write a Boolean expression for each labeled signal.
4. Continue level by level toward the outputs.
5. Substitute intermediate expressions to obtain outputs in terms of inputs.
6. Build a truth table and interpret the operation.

Analysis is more than algebra

Recognizing the function matters: the same equations may reveal an adder, comparator, code converter, or data selector.

Worked analysis: recognizing a full adder

Suppose analysis gives

$$F_1 = \overline{A}\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\overline{C} + ABC,$$

$$F_2 = AB + AC + BC.$$

Therefore,

$$\boxed{F_1 = A \oplus B \oplus C}, \qquad \boxed{F_2 = \mathrm{maj}(A, B, C)}.$$

A	B	C	F_2	F_1
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

F_1 is the **sum**; F_2 is the **carry**.

Design procedure: specification → circuit



Logical correctness

- interpret the verbal requirements;
- account for unused input combinations;
- verify every required output.

Implementation quality

- gate and input count;
- logic depth and propagation delay;
- fan-out, wiring, power, and available cells.

Design example: BCD to excess-3

The input $ABCD$ represents decimal digits 0–9 in BCD. The output $wxyz$ is the binary value of the digit plus 3.

digit	BCD	$\longrightarrow$	excess-3	value
0	0000	$\longrightarrow$	0011	3
1	0001	$\longrightarrow$	0100	4
2	0010	$\longrightarrow$	0101	5
3	0011	$\longrightarrow$	0110	6
4	0100	$\longrightarrow$	0111	7
5	0101	$\longrightarrow$	1000	8
6	0110	$\longrightarrow$	1001	9
7	0111	$\longrightarrow$	1010	10
8	1000	$\longrightarrow$	1011	11
9	1001	$\longrightarrow$	1100	12

Use the don't-cares

Inputs 1010 through 1111 are invalid BCD. Mark them X in the Karnaugh maps and use them when they simplify the circuit.

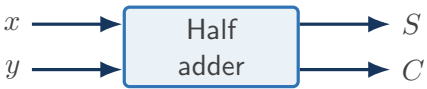
One factored solution is

$$z = \overline{D},$$
$$y = CD + \overline{CD},$$
$$x = \overline{B}(C + D) + B\overline{(C + D)},$$
$$w = A + B(C + D).$$

Half adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$\boxed{S = x \oplus y}$, $\boxed{C = xy}$



A half adder adds two bits, but it has **no carry input**.

Full adder

A full adder combines two operand bits and an input carry:

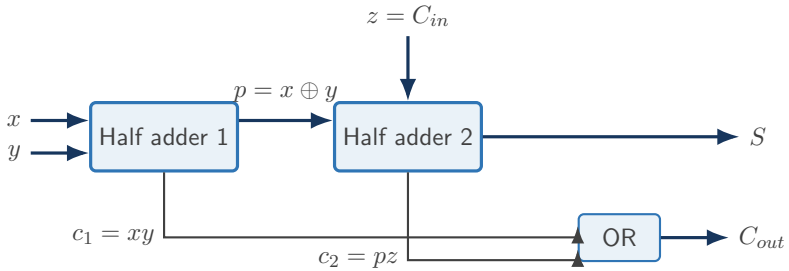
$$S_i = A_i \oplus B_i \oplus C_i,$$
$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i.$$

Interpretation

S_i is 1 for an odd number of asserted inputs.
C_{i+1} is 1 when at least two inputs are asserted.

A_i	B_i	C_i	C_{i+1}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Hierarchical full-adder construction

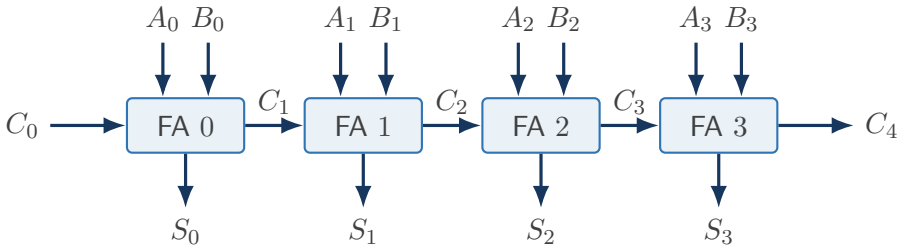


$$S = (x \oplus y) \oplus z, \qquad C_{out} = xy + z(x \oplus y).$$

Why hierarchy matters

Reusing verified cells exposes the regular structure needed for larger arithmetic circuits.

The n-bit ripple-carry adder



Structure

Cascade n full adders; connect each carry-out to the next carry-in.

Cost

The final result must wait for the worst-case carry to ripple across the chain.

Carry propagation and lookahead

Define a **propagate** and a **generate** signal at every bit:

$$P_i = A_i \oplus B_i, \qquad G_i = A_i B_i.$$

Then

$$S_i = P_i \oplus C_i, \qquad C_{i+1} = G_i + P_i C_i.$$

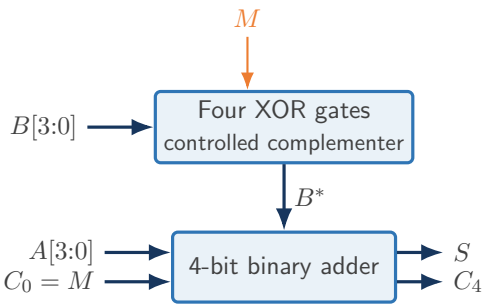
Expanding the recursion gives carries directly from C_0:

$$\begin{aligned} C_1 &= G_0 + P_0 C_0, \\ C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0, \\ C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0. \end{aligned}$$

Speed–area tradeoff

Lookahead creates all carries in parallel with more hardware; ripple carry uses less hardware but delay grows with word length.

One circuit for addition and subtraction



$$B_i^* = B_i \oplus M, \qquad C_0 = M$$

M	B^*	operation
0	B	$A + B$
1	$\overline{B}$	$A + \overline{B} + 1 = A - B$

XOR gates act as **controlled inverters**; the carry-in supplies the $+1$ for two's complementation.

Overflow: unsigned and signed interpretations

Unsigned arithmetic

- Addition overflow is the final carry C_n.
- For $A - B$ using two's complement, a borrow is indicated by $\overline{C}_n$.

Signed two's-complement arithmetic

The final carry alone does not indicate overflow.

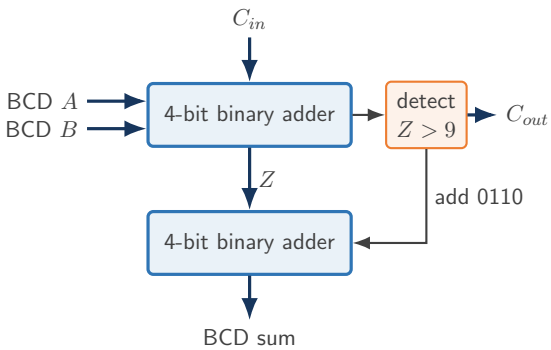
$$\boxed{V = C_n \oplus C_{n-1}}$$

Equivalent rule: equal-sign operands produce a result with the wrong sign.

Example in 8 bits

$70 + 80 = 150$ lies outside $[-128, 127]$. The stored bit pattern appears negative, and $V = 1$ flags the invalid signed result.

BCD addition: correct invalid binary sums



If the first adder produces $KZ_8Z_4Z_2Z_1$, correction is required when

$$\boxed{C = K + Z_8Z_4 + Z_8Z_2}.$$

- 0–9: keep the binary sum;
- 10–19: add 0110_2;
- C becomes the decimal carry.

Binary multiplication

$$
\begin{array}{r}
 B_1 B_0 \\
\times\ A_1 A_0 \\
\hline
A_0 B_1\ A_0 B_0 \\
+\ A_1 B_1\ A_1 B_0\ 0 \\
\hline
C_3 C_2 C_1 C_0
\end{array}
$$

Each partial-product bit is an AND:

$$p_{ij} = A_i B_j.$$

- Form one partial product for each multiplier bit.
- Shift partial products by their bit position.
- Add the aligned rows with half and full adders.

General resource pattern

A J-bit multiplier by a K-bit multiplicand uses JK AND gates and $J - 1$ stages of K-bit addition, producing up to $J + K$ product bits.

Magnitude comparator

Compare $A = A_{n-1} \cdots A_0$ and $B = B_{n-1} \cdots B_0$ from the MSB downward. Define per-bit equality

$$x_i = A_i \odot B_i = A_i B_i + \overline{A}_i \overline{B}_i.$$

For four bits,

$$(A = B) = x_3 x_2 x_1 x_0,$$

$$(A > B) = A_3 \overline{B}_3 + x_3 A_2 \overline{B}_2 + x_3 x_2 A_1 \overline{B}_1 + x_3 x_2 x_1 A_0 \overline{B}_0.$$

Priority is built into the expression

A lower bit matters only when every more-significant pair is equal. The expression for $(A < B)$ follows by exchanging A and B.

Decoder: one code word, one asserted output

An n-to-2^n decoder generates the minterms of its inputs. For a 3-to-8 decoder:

$$D_0 = \overline{x}\overline{y}\overline{z}, \qquad D_1 = \overline{x}\overline{y}z,$$
$$D_2 = \overline{x}y\overline{z}, \qquad \vdots$$
$$D_7 = xyz.$$

x	y	z	asserted output
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

Exactly one output is active: **one-hot decoding**.

Decoder enable, demultiplexing, and expansion

Enable

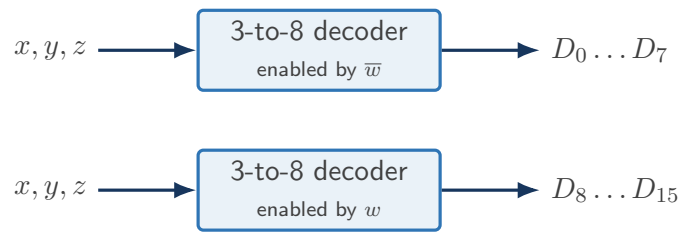
An enable input activates or disables the entire decoder. Either polarity may be used.

Demultiplexer

Treat enable as data and the decoder inputs as selects: one input is routed to one output.

Expansion

Use a high-order input to enable one of several smaller decoders.

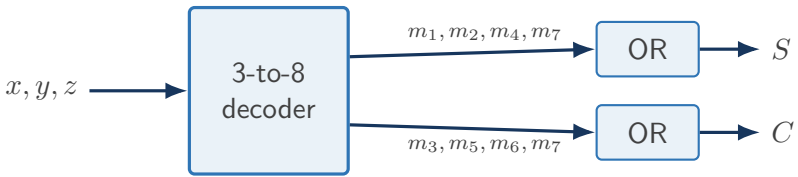


Implementing logic with a decoder

A decoder supplies every minterm. External OR gates collect the minterms required by each output.

Full-adder example:

$$S(x, y, z) = \Sigma m(1, 2, 4, 7), \qquad C(x, y, z) = \Sigma m(3, 5, 6, 7).$$



Encoder and its ambiguity

An encoder maps one of up to 2^n asserted inputs to an n-bit code. For an 8-to-3 octal encoder:

$$z = D_1 + D_3 + D_5 + D_7,$$
$$y = D_2 + D_3 + D_6 + D_7,$$
$$x = D_4 + D_5 + D_6 + D_7.$$

Multiple active inputs

Ordinary encoder output is ambiguous when more than one input is 1.

No active input

Output 000 cannot distinguish "no input" from $D_0 = 1$.

Both limitations motivate a **priority encoder with a valid bit**.

Four-input priority encoder

Let D_3 have the highest priority and D_0 the lowest.

D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

$$x = D_2 + D_3,$$
$$y = D_3 + D_1\overline{D}_2,$$
$$V = D_0 + D_1 + D_2 + D_3.$$

Meaning of V

$V = 1$ says that the encoded output is valid. Lower-priority inputs are ignored whenever a higher-priority input is active.

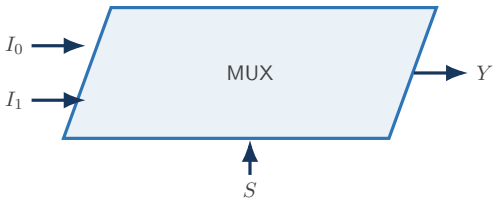
Multiplexer: a digitally controlled switch

A 2^n-to-1 multiplexer has

- 2^n data inputs;
- n selection inputs;
- one data output.

For a 2-to-1 MUX,

$$\boxed{Y = \overline{S} I_0 + S I_1}.$$



S	Y
0	I_0
1	I_1

Four-to-one multiplexer

$$Y = I_0\overline{S}_1\overline{S}_0 + I_1\overline{S}_1S_0 + I_2S_1\overline{S}_0 + I_3S_1S_0.$$

Selection inputs are decoded internally so that exactly one data path reaches the output.

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Parallel MUX units sharing select lines can switch an entire word or bus.

Implementing a Boolean function with a MUX

Implement

$$F(x, y, z) = \Sigma m(1, 2, 6, 7)$$

using x, y as the select inputs. Evaluate F as a function of the remaining variable z:

x	y	F as z varies	data input
0	0	$F = z$	$I_0 = z$
0	1	$F = \overline{z}$	$I_1 = \overline{z}$
1	0	$F = 0$	$I_2 = 0$
1	1	$F = 1$	$I_3 = 1$

General rule

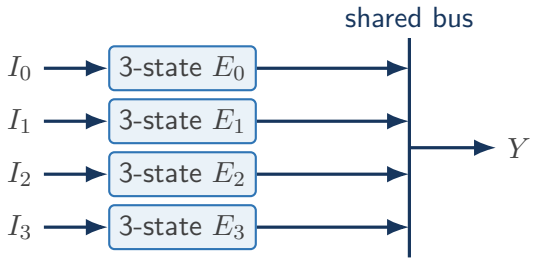
An n-variable function can be implemented with $n - 1$ select lines; each data input becomes 0, 1, the remaining variable, or its complement.

Three-state gates and shared buses

A three-state buffer produces

$$Y = \begin{cases} A, & C = 1, \\ Z, & C = 0, \end{cases}$$

where Z is a high-impedance state, electrically similar to disconnection.



Contention rule

At most one buffer may drive the shared line at a time. A decoder is a natural way to generate mutually exclusive enables.

Three Verilog modeling styles

Gate level

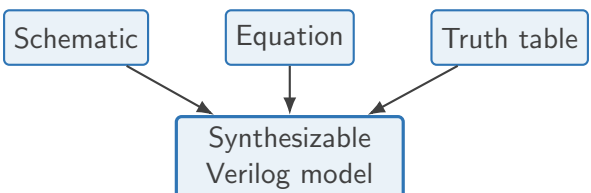
Instantiate primitives or modules. Mirrors a schematic and exposes structure.

Dataflow

Use continuous `assign` statements. Mirrors Boolean equations.

Behavioral

Use procedural `always` blocks. Mirrors algorithms or truth tables.



Gate-level and hierarchical modeling

```
module half_adder(
  output S, C,
  input  x, y
);
  xor (S, x, y);
  and (C, x, y);
endmodule
```

```
module full_adder(
  output S, C,
  input  x, y, cin
);
  wire p, c1, c2;
  half_adder HA1(p,  c1, x, y);
  half_adder HA2(S,  c2, p, cin);
  or (C, c1, c2);
endmodule
```

Instantiation rule

Module definitions cannot be nested, but instances of one module can appear inside another. This creates a reusable design hierarchy.

Dataflow modeling

```
module binary_adder(
  output [3:0] Sum,
  output       C_out,
  input  [3:0] A, B,
  input        C_in
);
  assign {C_out, Sum} = A + B + C_in;
endmodule
```

```
module mag_compare(
  output A_lt_B, A_eq_B, A_gt_B,
  input [3:0] A, B
);
  assign A_lt_B = (A < B);
  assign A_eq_B = (A == B);
  assign A_gt_B = (A > B);
endmodule
```

Dataflow models state *what* function is required; synthesis chooses a gate-level implementation.

Conditional operator: a MUX in one line

```
module mux_2x1_df(
  output m_out,
  input  A, B, select
);
  assign m_out = select ? A : B;
endmodule
```

```
condition ?  true_expression :  false_expression
```

Hardware interpretation

The conditional operator synthesizes naturally to selection logic. It can also be nested, but deep nesting can hide priority and increase logic depth.

Behavioral combinational modeling

```
module mux_4x1_beh(
  output reg m_out,
  input      in_0, in_1, in_2, in_3,
  input [1:0] select
);
  always @(*) begin
    case (select)
      2'b00: m_out = in_0;
      2'b01: m_out = in_1;
      2'b10: m_out = in_2;
      2'b11: m_out = in_3;
      default: m_out = 1'bx;
    endcase
  end
endmodule
```

Avoid accidental latches

Include every input in the sensitivity list (`@(*)` does this) and assign the output on every possible control path.

Three-state logic in Verilog

```
module mux_tri(
  output tri m_out,
  input      A, B, select
);
  bufif1 (m_out, A, select); // drive A when select = 1
  bufif0 (m_out, B, select); // drive B when select = 0
endmodule
```

- Verilog uses four logic states: `0`, `1`, `x` (unknown), and `z` (high impedance).
- A `tri` net permits multiple drivers and resolves their values in simulation.
- Mutually exclusive enables are essential to avoid bus contention.

A simple self-checking test bench

```
module test_mux;
  reg A, B, select;
  wire Y;
  integer i;
  mux_2x1_df dut(Y, A, B, select);
  initial begin
    for (i = 0; i < 8; i = i + 1) begin
      {select, A, B} = i; #10;
      if (Y !== (select ? A : B))
        $display("ERROR at vector %b", {select,A,B});
    end
    $finish;
  end
endmodule
```

Verification objective

For a small combinational block, apply all input combinations and compare every observed output with the specification.

Functional versus timing verification

Functional verification

- ignores physical gate delays;
- asks whether the logic matches the truth table;
- exhaustive testing is feasible for small n.

Timing verification

- includes propagation delays;
- checks when outputs become valid;
- exposes hazards and critical paths.

Correct logic at the wrong time can still be a system failure.

Putting the standard blocks together

Block	Purpose	Central design idea
Adder/subtractor	Arithmetic transformation	Cascade cells; control complementation with XOR
Comparator	Establish relative magnitude	Let the most-significant unequal pair decide
Decoder	Code to one-hot outputs	Generate minterms; add enable for control/expansion
Encoder	One-hot inputs to code	Add priority and a valid bit to resolve ambiguity
Multiplexer	Many data sources to one destination	Decode selects and enable one data path
Three-state buffer	Share a physical line	Only one enabled driver at a time

Design checklist

1. Is the circuit truly combinational—no state and no feedback?
2. Are input/output meanings, widths, and active levels explicit?
3. Does the truth table cover valid, invalid, and don't-care combinations?
4. Are the equations simplified for the actual design objective?
5. Can standard blocks replace a large flat gate network?
6. Is the worst-case propagation path acceptable?
7. Does the HDL assign every output on every path?
8. Has simulation checked normal cases, boundaries, and invalid inputs?

Quick knowledge check

1. Why is C_n sufficient for unsigned addition overflow but not for signed addition?
2. What four values may appear on a MUX data input when one variable is left out of the select set?
3. How does an enable input let two 3-to-8 decoders form a 4-to-16 decoder?
4. Why does carry lookahead reduce delay, and what does it cost?
5. What coding mistake in an `always` block can infer unintended storage?

Suggested exercise

Design a 4-bit adder–subtractor, write a dataflow model, and build a test bench that checks all operand pairs for both modes, including signed overflow.

Chapter 4 complete

Specify $\rightarrow$ simplify $\rightarrow$ implement $\rightarrow$ verify

Next: synchronous sequential logic